Gaussian Processes

In this exercise, you will implement Gaussian process regression and apply it to a toy and a real dataset. We use the notation used in the paper "Rasmussen (2005). Gaussian Processes in Machine Learning" linked on ISIS.

Let us first draw a training set $X = (x_1, \ldots, x_n)$ and a test set $X_\star = (x_1^\star, \ldots, x_m^\star)$ from a d-dimensional input distribution. The Gaussian Process is a model under which the real-valued outputs $\mathbf{f} = (f_1, \ldots, f_n)$ and $\mathbf{f}_\star = (f_1^\star, \ldots, f_m^\star)$ associated to X and $X_\star$ follow the Gaussian distribution:

$$\begin{bmatrix} \mathbf{f} \\ \mathbf{f}_\star \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix}, \begin{bmatrix} \Sigma & \Sigma_\star \\ \Sigma_\star^\top & \Sigma_{\star\star} \end{bmatrix} \right)$$

where

$$\begin{aligned} \Sigma &= k(X, X) + \sigma^2 I \\ \Sigma_\star &= k(X, X_\star) \\ \Sigma_{\star\star} &= k(X_\star, X_\star) + \sigma^2 I \end{aligned}$$

and where $k(\cdot, \cdot)$ is the Gaussian kernel function. (The kernel function is implemented in `utils.py`.) Predicting the output for new data points $X_\star$ is achieved by conditioning the joint probability distribution on the training set. Such conditional distribution called posterior distribution can be written as:

$$\mathbf{f}_\star | \mathbf{f} \sim \mathcal{N}(\underbrace{\Sigma_\star^\top \Sigma^{-1} \mathbf{f}}_{\boldsymbol{\mu}_\star}, \underbrace{\Sigma_{\star\star} - \Sigma_\star^\top \Sigma^{-1} \Sigma_\star}_{C_\star}) \tag{1}$$

Having inferred the posterior distribution, the log-likelihood of observing for the inputs $X_\star$ the outputs $\mathbf{y}_\star$ is given by evaluating the distribution $\mathbf{f}_\star | \mathbf{f}$ at $\mathbf{y}_\star$:

$$\log p(\mathbf{y}_\star | \mathbf{f}) = -\frac{1}{2}(\mathbf{y}_\star - \boldsymbol{\mu}_\star)^\top C_\star^{-1} (\mathbf{y}_\star - \boldsymbol{\mu}_\star) - \frac{1}{2} \log |C_\star| - \frac{m}{2} \log 2\pi \tag{2}$$

where $|\cdot|$ is the determinant. Note that the likelihood of the data given this posterior distribution can be measured both for the training data and the test data.

Part 1: Implementing a Gaussian Process (30 P)

Tasks:

- **Create a class `GP_Regressor` that implements a Gaussian process regressor and has the following three methods:**

- **`def __init__(self,Xtrain,Ytrain,width,noise):`** Initialize a Gaussian process with noise parameter σ and width parameter w. The variable `Xtrain` is a two-dimensional array where each row is one data point from the training set. The Variable `Ytrain` is a vector containing the associated targets. The function must also precompute the matrix Σ^{-1} for subsequent use by the method `predict()` and `loglikelihood()`.

- **`def predict(self,Xtest):`** For the test set $X_\star$ of m points received as parameter, return the mean vector of size m and covariance matrix of size $m \times m$ of the corresponding output, that is, return the parameters $(\boldsymbol{\mu}_\star, C_\star)$ of the Gaussian distribution $\mathbf{f}_\star | \mathbf{f}$.

- **`def loglikelihood(self,Xtest,Ytest):`** For a data set $X_\star$ of m test points received as first parameter, return the loglikelihood of observing the outputs $\mathbf{y}_\star$ received as second parameter.

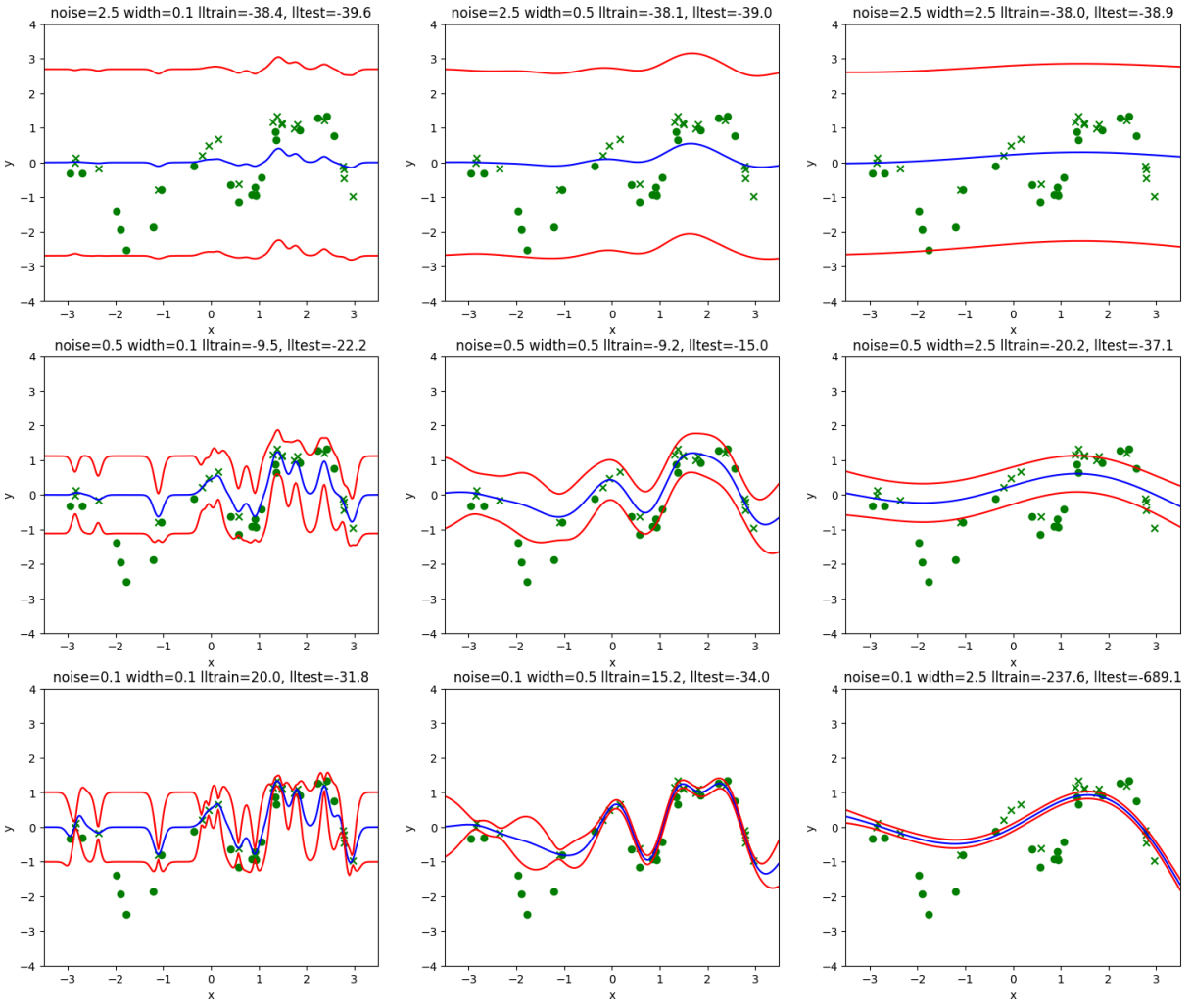
In [1]:
```python
import numpy as np
import utils


class GP_Regressor(object):
    def __init__(self, Xtrain, Ytrain, width, noise):

        self.Xtrain = np.asarray(Xtrain)
        self.Ytrain = np.asarray(Ytrain).reshape(-1)
        self.width = width
        self.noise = noise

        # K(X, X)
        K = utils.gaussianKernel(self.Xtrain, self.Xtrain, self.width)

        # Σ = K + σ^2 I
        n = self.Xtrain.shape[0]
        self.Sigma = K + (self.noise ** 2) * np.eye(n)

        # Σ^{-1}
        self.Sigma_inv = np.linalg.inv(self.Sigma)

    def predict(self, Xtest):

        Xtest = np.asarray(Xtest)
        m = Xtest.shape[0]

        # Σ* = k(X, X*)
        K_s = utils.gaussianKernel(self.Xtrain, Xtest, self.width)  # (n, m)

        # Σ** = k(X*, X*) + σ^2 I
        K_ss = utils.gaussianKernel(Xtest, Xtest, self.width)
        K_ss = K_ss + (self.noise ** 2) * np.eye(m)

        # μ* = Σ*^T Σ^{-1} y
        alpha = self.Sigma_inv @ self.Ytrain        # (n,)
```

```
        mean = K_s.T @ alpha                          # (m,)

        # C* = Σ** - Σ*^T Σ^{-1} Σ*
        cov = K_ss - K_s.T @ self.Sigma_inv @ K_s    # (m, m)

        return mean, cov

    def loglikelihood(self, Xtest, Ytest):

        Ytest = np.asarray(Ytest).reshape(-1)
        mean, cov = self.predict(Xtest)
        m = Ytest.shape[0]

        diff = Ytest - mean  # (m,)

        # C*^{-1}
        cov_inv = np.linalg.inv(cov)

        # (y - μ)^T C^{-1} (y - μ)
        quad = diff.T @ (cov_inv @ diff)

        # log |C*|
        sign, logdet = np.linalg.slogdet(cov)
        if sign <= 0:

            jitter = 1e-9 * np.eye(m)
            sign, logdet = np.linalg.slogdet(cov + jitter)

        loglik = -0.5 * (quad + logdet + m * np.log(2 * np.pi))
        return loglik
```

- **Test your implementation by running the code below (it visualizes the mean and variance of the prediction at every location of the input space) and compares the behavior of the Gaussian process for various noise parameters σ and width parameters w.**

In [2]:
```
import utils,datasets,numpy
import matplotlib.pyplot as plt
%matplotlib inline

# Open the toy data
Xtrain,Ytrain,Xtest,Ytest = utils.split(*datasets.toy())

# Create an analysis distribution
Xrange = numpy.arange(-3.5,3.51,0.025)[:,numpy.newaxis]

f = plt.figure(figsize=(18,15))

# Loop over several parameters:
for i,noise in enumerate([2.5,0.5,0.1]):
    for j,width in enumerate([0.1,0.5,2.5]):

        # Create Gaussian process regressor object
        gp = GP_Regressor(Xtrain,Ytrain,width,noise)
```

```
        # Compute the predicted mean and variance for test data
        mean,cov = gp.predict(Xrange)
        var = cov.diagonal()

        # Compute the log-likelihood of training and test data
        lltrain = gp.loglikelihood(Xtrain,Ytrain)
        lltest  = gp.loglikelihood(Xtest ,Ytest )

        # Plot the data
        p = f.add_subplot(3,3,3*i+j+1)
        p.set_title('noise=%.1f width=%.1f lltrain=%.1f, lltest=%.1f'%(noise
        p.set_xlabel('x')
        p.set_ylabel('y')
        p.scatter(Xtrain,Ytrain,color='green',marker='x') # training data
        p.scatter(Xtest,Ytest,color='green',marker='o')   # test data
        p.plot(Xrange,mean,color='blue')                  # GP mean
        p.plot(Xrange,mean+var**.5,color='red')           # GP mean + std
        p.plot(Xrange,mean-var**.5,color='red')           # GP mean - std
        p.set_xlim(-3.5,3.5)
        p.set_ylim(-4,4)
```



Part 2: Application to the Yacht Hydrodynamics Data Set (20 P)

In the second part, we would like to apply the Gaussian process regressor that you have implemented to a real dataset: the Yacht Hydrodynamics Data Set available on the UCI repository at the webpage http://archive.ics.uci.edu/ml/datasets/Yacht+Hydrodynamics. As stated on the web page, the input variables for this regression problem are:

1. Longitudinal position of the center of buoyancy
2. Prismatic coefficient
3. Length-displacement ratio
4. Beam-draught ratio
5. Length-beam ratio
6. Froude number

and we would like to predict from these variables the residuary resistance per unit weight of displacement (last column in the file `yacht_hydrodynamics.data`).

Tasks:

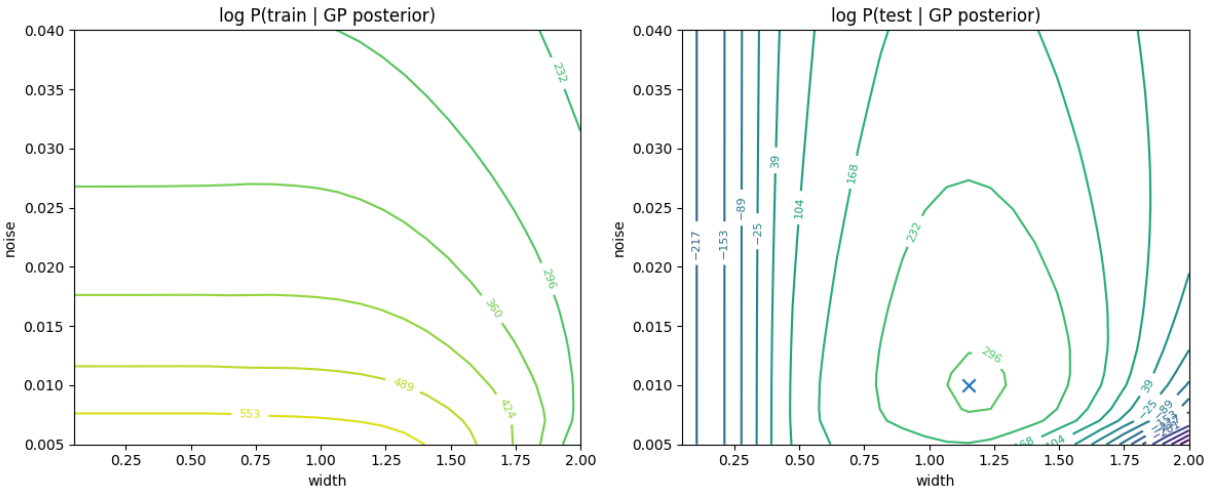
- **Load the data using `datasets.yacht()` and partition the data between training and test set using the function `utils.split()`. Standardize the data (center and rescale) so that each dimension of the training data and the labels have mean 0 and standard deviation 1 over the training set.**
- **Train several Gaussian processes on the regression task using various combinations of width and noise parameters.**
- **Draw two contour plots where the training and test log-likelihood are plotted as a function of the noise and width parameters. Choose suitable ranges of parameters so that the best parameter combination for the test set is in the plot. Use the same ranges and contour levels for the training and test plots. The contour levels can be chosen linearly spaced between e.g. 50 and the maximum log-likelihood value**

In [3]:
```
# train/test
X, Y = datasets.yacht()
Xtrain, Ytrain, Xtest, Ytest = utils.split(X, Y)

Xtrain = np.asarray(Xtrain)
Xtest  = np.asarray(Xtest)
Ytrain = np.asarray(Ytrain).reshape(-1)
Ytest  = np.asarray(Ytest).reshape(-1)


X_mean = Xtrain.mean(axis=0)
X_std  = Xtrain.std(axis=0)
X_std[X_std == 0] = 1.0

Xtrain_std = (Xtrain - X_mean) / X_std
```

```
Xtest_std  = (Xtest  - X_mean) / X_std

Y_mean = Ytrain.mean()
Y_std  = Ytrain.std()
if Y_std == 0:
    Y_std = 1.0

Ytrain_std = (Ytrain - Y_mean) / Y_std
Ytest_std  = (Ytest  - Y_mean) / Y_std

# noise / width
noise_vals = np.array([
    0.005, 0.007, 0.008, 0.010, 0.011, 0.013, 0.014, 0.016,
    0.017, 0.019, 0.020, 0.022, 0.023, 0.025, 0.026, 0.028,
    0.029, 0.031, 0.032, 0.034, 0.035, 0.037, 0.038, 0.040
])

width_vals = np.array([
    0.050, 0.135, 0.220, 0.304, 0.389, 0.474, 0.559, 0.643,
    0.728, 0.813, 0.898, 0.983, 1.067, 1.152, 1.237, 1.322,
    1.407, 1.491, 1.576, 1.661, 1.746, 1.830, 1.915, 2.000
])

print("Noise params:", " ".join(f"{v:0.3f}" for v in noise_vals))
print("Width params:", " ".join(f"{v:0.3f}" for v in width_vals))

# log-likelihood
ll_train = np.zeros((len(noise_vals), len(width_vals)))
ll_test  = np.zeros((len(noise_vals), len(width_vals)))

for i, noise in enumerate(noise_vals):
    for j, width in enumerate(width_vals):
        gp = GP_Regressor(Xtrain_std, Ytrain_std, width, noise)
        ll_train[i, j] = gp.loglikelihood(Xtrain_std, Ytrain_std)
        ll_test[i, j]  = gp.loglikelihood(Xtest_std,  Ytest_std)

best_i, best_j = np.unravel_index(np.argmax(ll_test), ll_test.shape)

# contour plot
vmin = min(ll_train.min(), ll_test.min())
vmax = max(ll_train.max(), ll_test.max())
levels = np.linspace(vmin, vmax, 20)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# train
cs1 = axes[0].contour(width_vals, noise_vals, ll_train, levels=levels)
axes[0].clabel(cs1, inline=1, fontsize=8)
axes[0].set_title("log P(train | GP posterior)")
axes[0].set_xlabel("width")
axes[0].set_ylabel("noise")

# test
cs2 = axes[1].contour(width_vals, noise_vals, ll_test, levels=levels)
axes[1].clabel(cs2, inline=1, fontsize=8)
axes[1].set_title("log P(test | GP posterior)")
```

```
axes[1].set_xlabel("width")
axes[1].set_ylabel("noise")

# test LL max value
axes[1].scatter(width_vals[best_j], noise_vals[best_i], marker="x", s=80)

plt.tight_layout()
plt.show()
```

```
Noise params: 0.005 0.007 0.008 0.010 0.011 0.013 0.014 0.016 0.017 0.019 0.
020 0.022 0.023 0.025 0.026 0.028 0.029 0.031 0.032 0.034 0.035 0.037 0.038
0.040
Width params: 0.050 0.135 0.220 0.304 0.389 0.474 0.559 0.643 0.728 0.813 0.
898 0.983 1.067 1.152 1.237 1.322 1.407 1.491 1.576 1.661 1.746 1.830 1.915
2.000
```



In [ ]: